Data Handling in R

2026-10-16

Abstract Corrections of and comments on the syllabus text, the example scripts, and answers to selected assignments are solicited. Requests for additions are welcome as well. Future students will appreciate your input! We acknowledge valuable contributions by former teachers Ozan Aksoy, Jos Dessens, Wim Jansen, Peter van de Heijden, Peter Ligtig, David Macro, Vardan Barsegyan, and Weverthon Machado as well as by numerous students.

Table of contents

1 Introduction . . . 2
2 Getting to know and summarizing data . . . 2
2.1 R . . . 3
2.2 Assignments . . . 8
3 Documenting data . . . 9
3.1 R . . . 9
3.2 Assignments . . . 11
3.3 Answers . . . 11
3.3.a 1. Rename the variable `hrespnr` to `hid` and the variable `hrinc` to `hinc` in one command. . . . 11
3.3.b 2. The variable `sex` is coded numerically. Convert `sex` into a factor with meaningful labels. Use `male` and `female` as category labels. . . . 11
3.3.c 3. The variables `sexrol1` and `sexrol2` measure family attitudes. Convert both variables into factors with the following category labels: fully disagree, slightly disagree, indifferent, slightly agree, fully agree. . . . 12
4 Creating new variables . . . 12
4.1 R . . . 12
4.2 Assignments . . . 16
4.2.a 1. Create dummy variables. . . . 16
4.2.b 2. Recoding a categorical variable. . . . 17
4.2.c 3. Recode a continuous variable into groups. . . . 17
4.2.d 4. Centering and standardizing variables. . . . 18
4.2.e 5. Creating scales . . . 18
5 Selecting cases and repeating analyses for groups . . . 19

5.1 R 19
5.2 Assignments 21

1 Introduction

- Contents via shortcode (see below)

2 Getting to know and summarizing data

In this part you will learn about the following basic tasks:

- Load a dataset from disk into computer memory.
- Explore the variables in the data.
- List values for (some) variables.
- Sort the observations on variables.
- The concept of "key variable(s)".
- Create frequency distributions of variables.
- Create cross tabulations of multiple variables.
- Produce a table of summary statistics (e.g., means, sd = standard deviation) for a series of variables.
- Produce a table of correlations of quantitative variables.

The concept of key variables may be new to you. A key consists of one or more variables that uniquely identify observations.

In social science datasets, we often use single observations to represent all variables measured on subjects. Each person is then a row in the data matrix. Such datasets often contain a respondent number, say `id`, usually integer valued. The values of `id` are usually unique, and then `id` qualifies as a key. If you know a subject's value of `id`, you can find the corresponding row in the data matrix, and hence know all the subject's characteristics.

If a value of `id` occurs more than once, obviously, the value of `id` does not identify a unique subject, and `id` is not a key. Observe that `id` need not take all possible values. For instance, if your `id` consists of 9 digits, likely your dataset does not contain all possible numbers of 9 digits. Many possible values of `id` will not be used. That is not a problem. `id` would still be a key if all values are distinct.

If `id` takes duplicate values, we may form a key by considering additional variables. For instance, think of a household survey with one or two adults. Observations are people nested in households. The variable `hid`, household number, would not be a key. The combination of `hid` and `sex` might be a key if the sample comprises single people and different-sex couples, but not if the sample also includes same-sex couples, cohabiting siblings, etc. In the latter case, the data may contain a variable indicating whether a person is the oldest adult in the household, and together with `hid` this variable may form a key. If you only have birth dates, the combination of `hid` and birth date

is usually a key, but you could have bad luck if your data include a household with two cohabiting twin sisters.

If a dataset does not contain a case identifier such as a respondent number, we strongly suggest you create one. Be sure to do this in a reproducible way. First put the observations in a specific order, by sorting on a series of variables that uniquely identify cases. In the example below, we assume that each observation has a unique pattern on the variables `city`, `age`, `edu`, and `sex`. Obviously, you should verify this assumption.

In R, you can check duplicates and create an identifier as follows:

```
df |>
  count(city, age, edu, sex) |>
  filter(n > 1)

df <- df |>
  arrange(city, age, edu, sex) |>
  mutate(respnr = row_number())
```

After sorting the cases, a new variable is created with the value 1 in the first observation, 2 in the second observation, 3 in the third observation, etc. This is one of the occasions in which you may want to save the dataset, but we urge you not to overwrite the original file.

2.1 R

You will learn the R functions listed below to describe and summarize data. Read the corresponding documentation pages when needed. Start with the description and examples. We do not expect you to learn all functions by heart, but you should get a rough idea of, and remember, what they can accomplish.

Function	Description
`read_csv()`	Load a CSV file into R
`read_dta()` / `read_sav()`	Load a Stata / SPSS file into R
`read_excel()`	Load an Excel file into R
`glimpse()`	Describe data contents
`skim()`	Describe data contents and summary statistics
`View()`	View observations and variables
`arrange()`	Sort observations
`table()`	One-way and two-way tables of frequencies
`prop.table()`	Convert frequencies to proportions
`tabyl()`	Frequency tables with percentages

Function	Description
`chisq.test()`	Pearson's chi-square test for independence
`summarise()`	Summary statistics
`cor()`	Correlations (Pearson by default)
`count()`	Count observations by category or group
`select()`	Select variables
`filter()`	Select observations satisfying a condition
`slice()`	Select observations by row number

Examples:

Load a file into memory, using a relative path:

```
library(readr) # for opening CSV files
data <- read_csv("data/mydata.csv")

library(haven) # for opening Stata (.dta) and SPSS (.sav) files
data <- read_dta("data/mydata.dta")
data <- read_sav("data/mydata.sav")

library(readxl) # for opening Excel (.xlsx) files
data <- read_excel("data/mydata.xlsx")
```

Describe variables in the active dataset:

```
glimpse(data) # shows basic info about the dataset and
              # a overview of all variables

data |>
  select(edu, exp) |>
  glimpse() # compact overview of edu and exp

data |>
  select(starts_with("rat")) |>
  glimpse() # describe all variables starting with rat

skim(data) # summary statistics  (from package 'skimr')
View(data) # open data viewer in RStudio
```

List observations:

```
data # all variables and all observations

data |>
  select(edu, inc) # selection of variables

data |>
  filter(country == 4) # selection of observations
                       # (all rows in which variable 'country' equals 4)

data |>
  select(country, edu, inc) |>    # selection of variables
  slice(1:20)                     # selection of observations 1 to 20
```

Sort observations on one or more variables:

```
data |>
  arrange(edu) # sort by edu (increasing)

data |>
  arrange(edu, inc) # sort by edu and inc  (increasing)

data |>
  arrange(desc(edu)) # sort by edu (decreasing)
```

One-way tabulation, frequencies:

```
table(data$class) # one-way table

table(data$edu, useNA = "ifany") # one-way table, include missing values

# usign tabyl, from package 'janitor'
data |>
  tabyl(edu) # frequency table with missing values and percentages
```

Two-way tabulation of variables:

```
table(data$edu, data$class)

# using tabyl:
data |>
  tabyl(edu, class)
```

Row and column percentages:

```r
# row percentages:
table(data$edu, data$class) |>
  prop.table(margin = 1)
# column percentages:
table(data$edu, data$class) |>
  prop.table(margin = 2)

# using tabyl:
data |>
  tabyl(edu, class) |>
  adorn_percentages("row")

data |>
  tabyl(edu, class) |>
  adorn_percentages("columns") |>
  adorn_pct_formatting() # optional
```

In `prop.table`, the argument `margin = 1` computes proportions within rows, while `margin = 2` computes proportions within columns.

Pearson's chi-square test for independence:

```r
table(data$edu, data$class) |>
  chisq.test()
# Pearson's chi-square test for independence
```

The chi-square test checks whether two categorical variables are statistically independent.

Tabulation of three or more variables:

```r
# three variables
xtabs(~ edu + class + gender, data = data, addNA=TRUE)

data |>
  tabyl(edu, class, gender)

# four variables (not possible with tabyl)
xtabs(~ edu + class + gender + country, data = data, addNA=TRUE)

# It is also possible to obtain frequencies using count().
# But it is not printed as a table and it does not show missing combinations
data |>
  count(gender, edu, class)
```

```
data |>
  count(country, gender, edu, class)
```

Summary statistics:

```
# Summary statistics for income
data |>
  skim(income)

data |>
  summarise(
    mean_income = mean(income, na.rm = TRUE),
    sd_income = sd(income, na.rm = TRUE),
    min_income = min(income, na.rm = TRUE),
    max_income = max(income, na.rm = TRUE)
    )
```

Correlation matrix of two or more variables:

```
data |>
  select(edu, inc, age) |>
  cor(use = "complete.obs") # Pearson correlations
```

Count the number of observations meeting a condition (you will practice this again in Selecting cases):

```
# count observations with income above 50000:
data |>
  filter(income > 50000) |>
  nrow()

# count observations with income between 5000 and 50000, boundaries excluded:
data |>
  filter(income > 5000, income < 50000) |>
  nrow()

# count observations with income between 5000 and 50000, boundaries included:
data |>
 filter(income >= 5000, income <= 50000) |> # alternative: filter(between(income,
5000, 50000))
  nrow()

# count observations with income above 50000 and income not missing:
```

```
data |>
  filter(income > 50000, !is.na(income)) |>
  nrow()
```

Check that the variable `birthd` is a key:

```
data |>
  summarise(no_missing = all(!is.na(birthd))) # birthd is never missing

data |>
  count(birthd) |>
  filter(n > 1) # check duplicates in birthd
```

Check that `birthd` and `sex` form a composite key:

```
data |>
  summarise(no_missing = all(!is.na(birthd), !is.na(sex))) # birthd and sex are
never missing

data |>
  count(birthd, sex) |>
  filter(n > 1) # check duplicates in the combination of birthd and sex
```

Create a key:

```
data <- data |>
  arrange(birthd, sex) |>
  mutate(id = row_number()) # create id after sorting by birthd and sex
```

2.2 Assignments

In these assignments we use the dataset `sbs399.dta`.

1. How many observations and variables the dataset contains?
2. What is the variable label of `feduc`?
3. Display a frequency distribution of `educ`. What is the percentage of subjects with an education up to category 4? How many missing values are there in `educ`?
4. Select the variables `hrinc` and `pres`. Adjust the syntax to show these variables only for men among the initial 40 observations. Further adjust the syntax to restrict the sample to men with below median education (see previous question) among the initial 40 observations. Finally, show the variables `hrinc` and `pres` for men and women among the initial 5 observations and arrange the results by sex.

5. List values of `age` in increasing order. Next, list just the five smallest values, and list just the five largest values. Do not read these values from the data viewer. Give syntax to display the requested cases.
6. Create a two-way table of `feduc` rows and `meduc` columns. Add row and column percentages. Test for independence of `feduc` and `meduc` using Pearson's chi-square test.
7. What are the mean, standard deviation, median, minimum, and maximum value of `feduc`?
8. Verify that the number of subjects for whom the mother is higher educated than the father is 543. What is the count if you restrict to female subjects?
9. What are the Pearson correlations of `educ`, `feduc`, and `meduc`? Use the help system to find out how to compute Spearman's rank correlations.
10. Is it true that for any variable the values are above average for 50% of cases? If you are uncertain, check this with some variables. For an explanation, think about this example: what percentage of people have more than the average number of legs? 50%? Who are these people?
11. Is `hrespnr` a key, meaning that it is never missing and all values are unique? Why? Do not rely on the data viewer. Write syntax to obtain the information on which you base your answer. What about the pair of variables `hrespnr` and `sex`? Why? What about `hrespnr`, `sex`, and `age`, and `hrespnr`, `sex`, and `fage`?

3 Documenting data

In this short section you will learn about the following basic tasks:

- Rename variables and give them clear names.
- Use meaningful names for categorical values.
- Convert categorical variables to factors.
- Code missing values as `NA`.
- Change the order of variables in a dataset.

3.1 R

Learn the following R commands to document the data, variables, and values:

Function	Description
`rename()`	Rename variables
`mutate()`	Modify or create variables
`factor()`	Convert a variable to a factor
`na_if()`	Change a specific value to `NA`
`replace_na()`	Replace `NA` with a specific value
`relocate()`	Change the order of variables

Rename variables:

```
data <- data |>
  rename(educ = v101)
# rename v101 to educ

data <- data |>
  rename(sex = v1, edu = v2)
# rename multiple variables
```

Convert a categorical variable to a factor:

```
data <- data |>
  mutate(
    sex = factor(
      sex,
      levels = c(1, 2),
      labels = c("male", "female")
    )
  )
# assign meaningful labels to categories
```

Code missing values:

```
data <- data |>
  mutate(educ = na_if(educ, 99))
# change the value 99 to NA

data <- data |> mutate(across(c(educ, feduc), ~ na_if(.x, 9)))
# change the value 9 to NA in educ and feduc
```

Replace missing values with a numeric code:

```
data <- data |>
  mutate(educ = replace_na(educ, 99))
# change NA to 99
```

Change the order of variables:

```
data <- data |>
  relocate(hrespnr, sex, age, edu)
# put these variables first
```

Be careful when replacing missing values with numeric codes such as 99. In R, missing values should usually be coded as `NA`. Numeric missing-value codes can easily be treated as real values in analyses if they are not recoded properly.

3.2 Assignments

In these assignments we use the dataset `sbs399_unlabeled.dta`.

1. Rename the variable `hrespnr` to `hid` and the variable `hrinc` to `hinc` in one command.
2. The variable `sex` is coded numerically. Convert `sex` into a factor with meaningful labels. Use `male` and `female` as category labels.
3. The variables `sexrol1` and `sexrol2` measure family attitudes. Convert both variables into factors with the following category labels:
    1. fully disagree
    2. slightly disagree
    3. indifferent
    4. slightly agree
    5. fully agree

3.3 Answers

3.3.a 1. Rename the variable `hrespnr` to `hid` and the variable `hrinc` to `hinc` in one command.

```
data <- data |>
  rename(
    hid = hrespnr,
    hinc = hrinc
  )
```

3.3.b 2. The variable `sex` is coded numerically. Convert `sex` into a factor with meaningful labels. Use `male` and `female` as category labels.

```
data <- data |>
  mutate(
    sex = factor(
      sex,
      levels = c(1, 2),
      labels = c("male", "female")
    )
  )
```

3.3.c 3. The variables `sexrol1` and `sexrol2` measure family attitudes. Convert both variables into factors with the following category labels: fully disagree, slightly disagree, indifferent, slightly agree, fully agree.

```
data <- data |>
  mutate(
    sexrol1 = factor(
      sexrol1,
      levels = c(1, 2, 3, 4, 5),
      labels = c(
        "fully disagree",
        "slightly disagree",
        "indifferent",
        "slightly agree",
        "fully agree"
      )
    ),
    sexrol2 = factor(
      sexrol2,
      levels = c(1, 2, 3, 4, 5),
      labels = c(
        "fully disagree",
        "slightly disagree",
        "indifferent",
        "slightly agree",
        "fully agree"
      )
    )
  )
```

4 Creating new variables

In this part you will learn about the following basic tasks:

- Creating a binary (0/1) variable.
- Recoding a categorical variable.
- Recoding a continuous variable into groups.
- Centering and standardizing variables.
- Reverse coding of variables.
- Creating additive scales.

4.1 R

To create or modify existing variables in a dataset, we will use the function `mutate()`.

The table below shows useful functions for recoding variables. These functions are normally used within `mutate()`.

Function	Description
`if_else()`	Create/modify based on simple logical condition
`case_when()`	Create/modify based on multiple logical conditions
`recode_values()`	Recode values (`dplyr` version >= 1.2)
`replace_values()`	Replace values (`dplyr` version >= 1.2)
`across()`	Applies function to multiple columns

Examples:

Create a variable as a mathematical expression of existing variables:

```
data <- data |>
  mutate(
    hhinc = h_inc + w_inc,
    minc = (h_inc + w_inc) / 2,
    adiff = abs(age - page),
    inc2 = inc^2
  )
# sum, mean, absolute difference, and squared income (^ denotes "power")
```

Change negative values of an existing variable:

```
data <- data |>
  mutate(
    inc = if_else(inc < 0, 0, inc)
  )
# replace negative of values of inc by 0
```

Create a binary (“dummy”) variable:

```
# Create binary variable male.
# Suppose that sex is coded as female=1, male=2, and there are no missings

# The variable can be numerical (0/1)..
data |>
  mutate(
     male = if_else(sex == 2, 1, 0) # male = 1, female = 0. In thise case, we
could also simply do `male = sex - 1`
  )
```

```
# or logical (TRUE/FALSE)...
data |>
  mutate(
    male = (sex == 2) # male = TRUE, female = FALSE
  )
```

For many uses in R, having a binary variable as numeric (1/0) or logical (TRUE/FALSE) makes little difference. For example, R will treat TRUE/FALSE as 1/0 if you compute the mean of a logical vector. Logical binary variables can make code a bit more succint when they are used in other operations: with logical vectors you can write e.g `filter(male, employed)` instead of `filter(male == 1, employed == 1)`. But be aware that some mathematical and modelling functions expect only numerical variables.

In the code above, we presumed thhat there were no missing values. If there were missing values, they would be coded as `0`, which is obviously wrong. To deal with with missing values:

```
data <- data |>
  mutate(
    male = recode_values(sex,
                         1 ~ 0,
                         2 ~ 1) # Change 1 into 0, 2 into 1
  )
# By default, recode_values() assigns missing (NA) to any values we did not
specify. So any NAs would stay as NAs, and any values otehr than 1 and 2 would
become NA).
```

Recode a dummy variable:

```
data <- data |>
  mutate(
    female = 1 - male
  )
# male = 1 becomes female = 0 and vice versa
# NA values remain NA
```

Recode values into categories:

```
# create region variable
# 71, 72, 79 = 1
# 73 to 78 = 2
# all other valide (non NA) codes = 3
data <- data |>
```

```
  mutate(
    region = recode_values(country,
                           c(71, 72, 79)  ~ 1,
                           73:78          ~ 2,
                           NA ~ NA,
                           default = 3 # this applies to all other values not
speficied above.
                                       # Attention: if we had nto specified NA
~ NA above,
                                      # NA values would also have been recoded
to 3
    )
  )
```

You can also recode using `case_when()`, specifying logical conditions instead of original values on the lef-hand side. `case_when()` is thus more flexible (e.g. it can handle conditions based on multiple original variables, see below) but tends to more verbose.

```
# Same task as above, using case_when().
# Note that we explicitly refer to the original variable in each condition
data <- data |>
  mutate(
    region = case_when(
      country %in% c(71, 72, 79) ~ 1,
      country %in% 73:78         ~ 2,
      !is.na(country)            ~ 3,
      is.na(country)             ~ NA
    )
  )


# Recoding based on multiple existing variables
data <- data |>
  mutate(
    group = case_when(
      (male==1 & age < 18)  ~ "male_minor",
      (male==1 & age >= 18) ~ "male_adult"
      (male==0 & age < 18)  ~ "female_minor",
      (male==0 & age >= 18) ~ "female_adult"
     .default           ~ NA  # Everything else as NA. Note the dot in .default.
    )
  )
```

Create a standardized variable (z-scores):

```
data <- data |>
  mutate(
    zedu = (edu - mean(edu, na.rm = TRUE)) /
      sd(edu, na.rm = TRUE)
  )
# standardize edu
```

Create a rank variable:

```
data <- data |>
  mutate(
    rinc = rank(inc)
  )
# rank order statistics of income
```

With `across()` you can repeat operations across several columns:

```
# Create a new variable counting the number of variables with value == 2:
data <- data |>
  mutate(
    ngood = rowSums(
     across(good01:good05, ~ .x == 2), # for each variable good1 to good5, check
if the value is 2
      na.rm = TRUE
    )
  )

# Change 9999 to NA in all variables whose names start with "income"
data <- data |>
  mutate(
         across(starts_with("income"), ~if_else(.x == 9999, NA, .x))
         )
```

4.2 Assignments

4.2.a 1. Create dummy variables.

In the dataset `sbs399` the variable `class` marks the respondent's social class. How often is this variable non-missing?

We want to create a dummy variable for being of white collar. This variable, which we will call `white`, is 1 for subjects belonging to the white-collar class and 0 otherwise. We interpret missing

values in `class` to denote "unknown". Thus, the dummy `white` should also be missing if `class` is missing.

- Create this dummy variable using `if_else()`. Call it `white1`.
- Create the same dummy variable using `recode_values()`. Call it `white2`.
- Produce a cross-tabulation of the two newly created variables and verify that they are identical.

4.2.b 2. Recoding a categorical variable.

In the dataset `sbs399` the variable `polit` marks the political party the respondent intended to vote for in a national election.

a. Generate a new variable `pol_pref` in which political preference is expressed on a left-to-right scale:

```
1 = SP
2 = GroenLinks
3 = PvdA
4 = D66
5 = CDA
6 = VVD
7 = GPV
8 = RPF
9 = SGP
10 = Unie 55+
11 = AOV
12 = CD
```

b. Recode `pol_pref` into a new variable `pol_pref3` in which political preference is expressed in three categories:

```
1 = Left (SP, GroenLinks, PvdA, D66)
2 = Center (CDA)
3 = Right (all remaining parties)
```

4.2.c 3. Recode a continuous variable into groups.

We consider the hourly wage variable `hrinc`.

Generate a new variable `hrinc_quart` for the four quartiles of `hrinc`:

- hrinc_quart = 1 for the lowest 25%
- hrinc_quart = 2 for the next 25%
- hrinc_quart = 3 for the next 25%
- hrinc_quart = 4 for the highest 25%

Verify that the groups contain approximately equal numbers of observations.

4.2.d 4. Centering and standardizing variables.

We focus on `hrinc` again.

a. Create a variable `c_hrinc` that contains the centered hourly wage rate by subtracting the sample mean of `hrinc`. Check that:

- the mean of `c_hrinc` is approximately 0;
- the standard deviations of `c_hrinc` and `hrinc` are the same.

b. Create a variable `z_hrinc` that contains the standardized hourly wage rate by subtracting the mean of `hrinc` and dividing by its standard deviation. Check that:

- the mean of `z_hrinc` is approximately 0;
- the standard deviation of `z_hrinc` is approximately 1.

c. Create a variable `z2_hrinc` that standardizes hourly income separately for men and women. Within each sex, the mean of `z2_hrinc` should be approximately 0 and standard deviation approximately 1. Ensure that:

- you obtain one variable, not two;
- the variable is not missing unless `hrinc` itself is missing.

d. Apply a unit-range transformation to `hrinc`:

$$U(x) = \frac{x - \min(x)}{\max(x) - \min(x)}$$

- Verify that the transformed minimum equals 0 and the transformed maximum equals 1.

Which method do you prefer for income? And for number of hours worked per week?

4.2.e 5. Creating scales

We consider the five items `sexrol1` to `sexrol5` in the dataset `sbs399`. These items measure attitudes towards women's emancipation by (5 point) Likert items. We want to average the 5 items and treat it as a measurement of attitude, with a larger value indicating a more traditional attitude.

(a) Explain why it is not sensible to simply add the five items.

(b) You likely want to reverse one of the items. Which one? Construct a new variable for this reversed item using:

- a recoding approach with `recode_values()` (or `case_when()`);
- a mathematical expression. Verify that both constructed variables are identical.

(c) Create the attitude measure from the five items. Hint: use `rowMeans()`. What is the mean of the measure? Can you interpret this mean substantively?

(d) The data used so far are complete, meaning that respondents answered all items. In many surveys, however, respondents do not answer every question. This situation is illustrated by the dataset `sbs399_sexroles`. We will now discuss a simple ad hoc method to deal with this.

This method is quick-and-dirty; it used to be widely used, but theoretical statistics has yielded better methods. We will discuss these more advanced methods later on.

For now we simply take the mean of all valid item responses for each respondent. Thus, if a respondent answered items 1, 2, and 3, but not items 4 and 5, we calculate the scale score as the mean of items 1, 2, and 3. If a respondent answered all items, we take the mean across all items. If a respondent did not answer any of the items, the scale score should be missing as well.

In R, this can be accomplished using the argument `na.rm = TRUE` in `rowMeans()` (or in `mean()` used together with `rowwise()`), which computes the mean using all available values while ignoring missing values.

5 Selecting cases and repeating analyses for groups

In this part you will learn about selecting cases (observations).

Function	Description
`filter()`	Select observations satisfying a condition
`slice()`	Select observations by row number
`slice_head()`	Select the first observations
`slice_tail()`	Select the last observations
`group_by()`	Define groups for subsequent operations
`.by`	Apply an operation separately within groups

5.1 R

Examples:

Select observations using logical conditions:

```
data |>
  filter(sex == 1)
# select observations for which sex equals 1

data |>
  filter(sex == 1, edu > 3)
# select observations for which sex equals 1 AND edu is larger than 3

data |>
  filter(sex == 1 | edu >= 3)
# select observations for which sex equals 1 OR edu is at least 3
# the vertical bar means OR, so respondents satisfying either condition are
counted
```

```
data |>
  filter(edu != 3)
# select observations in which edu is different than 3


data |>
  filter((sex == 1 & edu > 3) | (sex == 0 & edu < 3))
# select observations in which one of this is true:
# 1. sex equals 1 and edu is larger than 3
# OR
# 2. sex equals 0 and education is lower than 3

data |>
  filter(age == max(age))
# select observation(s) whose age equals the maximum age in the dataset, i.e. the
# oldest person(s). Should use na.rm=TRUE is there are missing values in age
```

Select observations based on row number:

```
data |>
   slice(5)
# select 5th row

data |>
  slice(1:20)
# select rows 1 to 20

data |>
  slice_head(20)
# select first 20 observations (same as above)

data |>
  slice_tail(20)
# select the last 20 observations
```

The same can also be done with `filter()` and `row_number()`:

```
data |>
  filter(row_number() == 5)
# select 5th row

data |>
```

```
  filter(row_number() <= 20)
# select the first 20 observations

data |>
  filter(row_number() > (n() - 20))
# select the last 20 observations
# n() gives the total number of rows
```

Keep a subset of observations:

```
data_male <- data |>
  filter(male == 1)
# create a new dataset containing only observations for which male==1

data_first100 <- data |>
  slice(1:100)
# create a new dataset containing only the first 100 observations
```

The argument `.by` is to apply a filter within groups. This is useful when you want to compare an observation with other observations in each group, and filter based on that. You will learn more about grouped operations in Aggregating data. Suppose each observation belongs to a household, indicated by the id variable `hh_id`:

```
data |>
  filter(row_number() == 1, .by = hh_id)
# select the first row from *each household*. It is as if we create one dataset per
# household, selected the first row from each of them, them put them together again

data |>
  filter(age == max(age), .by = hh_id)
# select observations where age equals the maximum age *within each household*,
i.e.
#  the oldest in each household. Note that this can be more than one person
per household
# if they have the same age.
```

5.2 Assignments

In these assignments, you should use the dataset `sbs399`.

1. Using `filter()` and `nrow()`, count the number of observations in `sbs399`:
   - with age at most 30;
   - with age at least 20;

- with age between 20 and 30, boundaries included;
- with age between 20 and 30, boundaries excluded;
- males over 25;
- people who are either below 68 or female.

2. Select the subjects with age at most 30 and store this subset in a new object. Do not overwrite the original dataset `sbs399`.
   - Count the number of subjects of age at least 20 in this subset.
   - Count how many subjects of age at least 20 in this subset are men and how many are women.
3. Using the full dataset `sbs399` again.
   - Create a selection variable, say `touse`, that takes the value 1 for males over 30 and 0 otherwise.
   - For the selected sample:
     - create a two-way table for the respondent's education, `educ`, and the respondent's father's education, `feduc`.
     - compute the correlation between `educ` and `feduc`
4. What is the difference between the following R commands?

```
data |>
  select(-sex)

data |>
  filter(sex != 1)

data |>
  filter(sex == 1)
```

5. Using the household identifier `hrespnr` in the dataset `sbs399`, use `.by` or `group_by()` to:
   - Select the oldest people in households.
     - Bonus: try to figure out in how many households there are "ties", i.e. more than one person has the same maximum age.
   - List `sex` and `age` in increasing order of age in households where the age difference between the oldest and the youngets person is larger than 5, without creating new variables;
   - Drop people living alone in a household. How many cases remain in the dataset?